

State

Why React State is Immutable:

1. What is Immutability?

In programming, **immutable** means "unable to be changed after creation."

- When an object or array is immutable, you cannot alter its properties or elements directly.
- Instead of modifying the existing data structure, you **create a new copy** with the updated values.

2. Why React Demands Immutability

React's core mission is to keep your application's UI in sync with your application's state. To do this efficiently, React must quickly answer one question: **"Did the state change?"**

Reason A: Reference Equality Checks (Shallow Checking)

If React had to check every property of a deeply nested object to see if something changed, it would be extremely slow:

```
// Deep Comparison (Slow: O(N))  
if (prevUser.address.geo.lat !== newUser.address.geo.lat) { ... }
```

Instead, React performs a **Shallow Comparison** (using `Object.is` or simple `===` checks on the reference pointer in memory):

```
// Shallow Comparison (Instant: O(1))  
if (prevUser !== newUser) {  
  // Re-render component!  
}
```

- If you mutate an object in-place, the memory address (reference) stays the exact same. React sees `prevState === newState`, assumes nothing changed, and **skips re-rendering**.
- If you copy the object, it gets a **new memory address**. React sees `prevState !== newState` and instantly schedules a UI update.

3. Code Demonstration: Mutable vs. Immutable

Let's look at what happens in memory with two different approaches.

The Wrong Way (Direct Mutation)

```
import { useState } from 'react';

export default function Profile() {
  const [user, setUser] = useState({ name: 'Alex', age: 20 });

  const handleUpdate = () => {
    // WRONG: Mutating the state object directly
    user.age = 21;

    // Telling React to update, but passing the SAME object reference
    setUser(user);
  };

  return <button onClick={handleUpdate}>Age: {user.age}</button>;
}
```

What happens internally (Direct Mutation):

1. The user object is stored at memory address 0x111 (containing { name: 'Alex', age: 20 }).
2. user.age = 21 changes the age property directly inside address 0x111.
3. setUser(user) is called, passing the object at address 0x111.
4. React compares the current state pointer with the incoming pointer:
Object.is(0x111, 0x111) // Evaluates to true
5. Since the references are identical, **React bails out (does not trigger a re-render)**. The UI remains stuck showing "Age: 20" even though the variable changed.

The Correct Way (Immutable Update)

```
import { useState } from 'react';

export default function Profile() {
  const [user, setUser] = useState({ name: 'Alex', age: 20 });

  const handleUpdate = () => {
    // Correct: Creating a new copy with the spread operator (...)
    const updatedUser = { ...user, age: 21 };

    // Passing the NEW object reference to the setter
    setUser(updatedUser);
  };

  return <button onClick={handleUpdate}>Age: {user.age}</button>;
}
```

```

    setUser(updatedUser);
  };

  return <button onClick={handleUpdate}>Age: {user.age}</button>;
}

```

What happens internally (Immutable Update):

1. The initial user object is stored at memory address 0x111 ({ name: 'Alex', age: 20 }).
2. { ...user, age: 21 } allocates a **new object** in memory at a brand-new address, e.g., 0x222 ({ name: 'Alex', age: 21 }).
3. setUser(updatedUser) is called, passing the object at address 0x222.
4. React compares the pointers: Object.is(0x111, 0x222) // Evaluates to false
5. Since they are different, **React schedules a re-render**. The UI updates to show "Age: 21".

4. Visualizing References in Memory

You can write this simple breakdown on a whiteboard for your students:

Step	State Action	Value	Memory Reference Pointer	React Reaction
Start	Initial Render	{ name: 'Alex', age: 20 }	0x111 (Original)	-
A	Direct Mutation	{ name: 'Alex', age: 21 }	0x111 (Unchanged pointer)	No Render (References matched)
B	Immutable Update	{ name: 'Alex', age: 21 }	0x222 (New allocation)	Re-renders UI (Pointers differed)

Hooks

What are Hooks?

Hooks are special functions provided by React that add extra capabilities to functional components, such as state management, lifecycle handling, DOM access, and performance optimization, without using class components.

Why do we use Hooks?

Imagine you want to build a counter.

Without Hooks

None

```
function App() {  
  let count = 0;  
  return (  
    <>  
      <h1>{count}</h1>  
      <button>Increment</button>  
    </>  
  );  
}
```

Clicking the button won't update the UI because count is just a normal JavaScript variable. React doesn't know it changed.

Using useState

None

```
import { useState } from "react";

function App() {
  const [count, setCount] = useState(0);
  return (
    <>
      <h1>{count}</h1>

      <button onClick={() => setCount(count +
1)}>
        Increment
      </button>
    </>
  );
}
```

Now React knows when the state changes and automatically re-renders the component.

The Rules of Hooks

Before diving into individual hooks, you must adhere to the two rules that govern them:

1. **Only Call Hooks at the Top Level:** Don't call Hooks inside loops, conditions, or nested functions. This ensures Hooks are called in the same order on every render.
2. **Only Call Hooks from React Functions:** Call them from React function components or custom Hooks. Do not call them from regular JavaScript functions.

1. State Hooks

State hooks let a component "remember" information like user input or current status between renders.

useState

Manages a reactive state variable in a function component.

- **Use Cases:** Simple values, toggles, form fields, and objects that do not have complex nested update logic.
- **How it works:** Returns the current state and a function to update it. Calling the updater triggers a re-render.

Code Example

```
import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);

  return (
```

```

<div className="card">
  <p>Count: {count}</p>
  {/ Use updater function when new state depends on previous state /}
  <button onClick={() => setCount(prev => prev + 1)}>Increment</button>
  <button onClick={() => setCount(0)}>Reset</button>
</div>
);
}

```

[!IMPORTANT] State updates are batched and asynchronous. If you need to update state based on the previous state, always pass an updater function (e.g., `setCount(prev => prev + 1)`) instead of the direct value (`setCount(count + 1)`).

useReducer

An alternative to `useState` for managing complex state structures or state updates that depend on multiple actions.

- **Use Cases:** Complex state objects, list management (todo lists, carts), and state transitions matching state machines.
- **How it works:** Takes a reducer function (`state, action`) => `newState` and initial state. Returns current state and a dispatch function.

Code Example

```

import { useReducer } from 'react';

const initialState = { count: 0, status: 'Idle' };

function reducer(state, action) {
  switch (action.type) {
    case 'increment':
      return { ...state, count: state.count + 1, status: 'Incremented' };
    case 'decrement':

```

```

    return { ...state, count: state.count - 1, status: 'Decrement' };
  case 'reset':
    return initialState;
  default:
    throw new Error(`Unhandled action type: ${action.type}`);
  }
}

function CounterWithReducer() {
  const [state, dispatch] = useReducer(reducer, initialState);

  return (
    <div>
      <p>Count: {state.count} (Status: {state.status})</p>
      <button onClick={() => dispatch({ type: 'increment' })}>+</button>
      <button onClick={() => dispatch({ type: 'decrement' })}>-</button>
      <button onClick={() => dispatch({ type: 'reset' })}>Reset</button>
    </div>
  );
}

export default app;

```

//dispatch is a function returned by the useReducer Hook that is used to send an action to the reducer function. The reducer processes the action and returns the updated state.

2. Context Hooks

useContext

Subscribes to a React context without nesting a Consumer component.

- **Use Cases:** Global themes, user authentication details, language/locale, and application settings.
- **How it works:** Takes a context object (created with `React.createContext`) and returns the current context value. When the provider value changes, the component automatically re-renders.

Code Example

```
import { createContext, useContext, useState } from 'react';

const ThemeContext = createContext(null);

export default function App() {
  const [theme, setTheme] = useState('dark');
  return (
    <ThemeContext.Provider value={theme}>
      <Card />
      <button onClick={() => setTheme(t => t === 'dark' ? 'light' : 'dark')}>
        Toggle Theme
      </button>
    </ThemeContext.Provider>
  );
}

function Card() {
  const theme = useContext(ThemeContext);
  return (
    <div className={`card ${theme}`}>
      <p>The current theme is {theme}</p>
    </div>
  );
}
```

[!WARNING] `useContext` will always trigger a re-render when the context value changes, even if the component only uses a small part of that context. To optimize, split contexts or memoize children.

3. Ref Hooks

Refs allow components to hold mutable values that do not trigger re-renders, and directly access DOM elements.

useRef

Creates a mutable reference object whose `.current` property is persisted across renders.

- **Use Cases:** Referencing DOM nodes (focusing input, scrolling), storing timers/intervals, or keeping track of the previous state value.
- **How it works:** Returns a ref object. Mutating `.current` does **not** cause a re-render.

Code Example

```
import { useRef, useEffect } from 'react';

function AutoFocusInput() {
  const inputRef = useRef(null);
  const timerRef = useRef(null);

  useEffect(() => {
    // Focus the input on mount
    inputRef.current.focus();

    timerRef.current = setInterval(() => {
      console.log('Tick');
    }, 1000);

    return () => clearInterval(timerRef.current);
  }, []);
```

```
return <input ref={inputRef} placeholder="Type here..." />;
}
```

useImperativeHandle

Customizes the instance value that is exposed to parent components when using ref.

- **Use Cases:** Creating reusable components (like custom modal boxes, media players) that expose custom imperative APIs (like `open()`, `close()`, `play()`).
- **How it works:** Must be used with `forwardRef`. Defines functions/values attached to the parent ref.

Code Example

```
import { useRef, useImperativeHandle, forwardRef } from 'react';

const CustomInput = forwardRef((props, ref) => {
  const localInputRef = useRef();

  useImperativeHandle(ref, () => ({
    focusAndSelect: () => {
      localInputRef.current.focus();
      localInputRef.current.select();
    }
  }));

  return <input ref={localInputRef} {...props} />;
});

// Parent component usage:
function Parent() {
  const inputRef = useRef();
  return (
    <>
      <CustomInput ref={inputRef} defaultValue="Hello" />
    </>
  );
}
```

```
<button onClick={() => inputRef.current.focusAndSelect()}>
  Focus & Select
</button>
</>
);
}
```

4. Effect Hooks

Effects let a component connect to and synchronize with external systems (network, browser DOM, third-party libraries).

useEffect

Synchronizes a component with an external system.

- **Use Cases:** API calls, subscribing to events (websocket, resize), timers, and manipulating the DOM imperatively.
- **How it works:** Runs after browser paint. Accepts a function and a dependency array. Returns an optional cleanup function.

Code Example

```
import { useState, useEffect } from 'react';

function FetchUser({ userId }) {
  const [user, setUser] = useState(null);

  useEffect(() => {
    let isCancelled = false;

    async function loadData() {
      const response = await fetch(`https://api.example.com/users/${userId}`);
      const data = await response.json();
      if (!isCancelled) {
        setUser(data);
      }
    }
  }, [userId]);
}
```

```

    }
    loadData();

    // Cleanup function: runs before next effect execution or unmount
    return () => {
      isCancelled = true;
    };
  }, [userId]); // Dependency array: triggers only when userId changes

  return user ? <div>{user.name}</div> : <p>Loading...</p>;
}

```

[!TIP] Always return a cleanup function to prevent memory leaks and race conditions (e.g., setting state on unmounted components or outdated requests).

useLayoutEffect

Fires synchronously after all DOM mutations but **before** the browser paints the screen.

- **Use Cases:** Measuring layouts (width, height, element position) and adjusting styling or scroll position before the user sees it (prevents flickering).
- **How it works:** Identical signature to `useEffect`, but runs blocking.

Code Example

```

import { useLayoutEffect, useRef, useState } from 'react';

function Tooltip({ children }) {
  const [height, setHeight] = useState(0);
  const containerRef = useRef(null);

  useLayoutEffect(() => {

```



```
// Measure DOM height before paint
if (containerRef.current) {
  setHeight(containerRef.current.getBoundingClientRect().height);
}
}, []);

return (
  <div ref={containerRef} style={{ bottom: `${height}px` }}>
    {children}
  </div>
);
}
```

useInsertionEffect

Fires before any DOM mutations and before `useLayoutEffect`. It is intended specifically for CSS-in-JS libraries.

- **Use Cases:** Dynamic styling in styling libraries (like styled-components) to insert `<style>` tags before layout calculations.
- **How it works:** Runs synchronously before DOM updates. Does not have access to refs.

5. Performance Hooks

Performance hooks optimize the rendering behavior by memoizing computations or deferring slow updates.

useMemo

Caches the result of a calculation between re-renders.

- **Use Cases:** Filtering lists, parsing large payloads, and preventing expensive derived-state calculations on every render.
- **How it works:** Computes the value only when dependencies change. Otherwise, returns the cached result.

Code Example

```
import { useMemo, useState } from 'react';

function ProductList({ items, filterText }) {
  const filteredItems = useMemo(() => {
    console.log('Filtering items...'); // Runs only if items or filterText change
    return items.filter(item => item.name.includes(filterText));
  }, [items, filterText]);

  return (
    <ul>
      {filteredItems.map(item => <li key={item.id}>{item.name}</li>)}
    </ul>
  );
}
```

useCallback

Caches a function definition between re-renders.

- **Use Cases:** Passing callback functions to optimized child components (React.memo) to prevent them from re-rendering due to reference changes.
- **How it works:** Returns the exact same function reference until dependencies change.

Code Example

```

import { useCallback, useState, memo } from 'react';

// Optimized child component
const Button = memo(({ onClick, label }) => {
  console.log(`Rendered: ${label}`);
  return <button onClick={onClick}>{label}</button>;
});

function Container() {
  const [count, setCount] = useState(0);

  // Reference does not change on count updates
  const handleAction = useCallback(() => {
    console.log('Button clicked');
  }, []);

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(c => c + 1)}>Increment Count</button>
      <Button onClick={handleAction} label="Action Button" />
    </div>
  );
}

```

useTransition

Lets you update the state without blocking the UI.

- **Use Cases:** Tab switching, live search filters, and updates that generate heavy rendering workflows.
- **How it works:** Returns an `isPending` state and a `startTransition` function. State updates inside `startTransition` are marked as non-blocking transitions and can be interrupted by user actions like typing.

Code Example

```
import { useState, useTransition } from 'react';

function SearchResults() {
  const [query, setQuery] = useState("");
  const [list, setList] = useState([]);
  const [isPending, startTransition] = useTransition();

  const handleChange = (e) => {
    const value = e.target.value;
    setQuery(value); // High priority: update input instantly

    // Transition (Low priority): process list filter without locking input
    startTransition(() => {
      const items = Array.from({ length: 20000 }, (_, i) => `${value} item ${i}`);
      setList(items);
    });
  };

  return (
    <div>
      <input type="text" value={query} onChange={handleChange} />
      {isPending ? <p>Loading list...</p> : (
        <ul>
          {list.slice(0, 100).map((item, i) => <li key={i}>{item}</li>)}
        </ul>
      )}
    </div>
  );
}
```

useDeferredValue

Lets you defer updating a part of the UI. Similar to debounce/throttle but managed natively by React's scheduler.

- **Use Cases:** Slow, expensive lists that update as typing occurs.
- **How it works:** Returns a deferred version of a value that trails the original value. During a transition, React will try to reuse the old value and then update it in the background.

Code Example

```
import { useState, useDeferredValue, useMemo } from 'react';

function SearchDashboard() {
  const [text, setText] = useState("");
  const deferredText = useDeferredValue(text);

  // Computed using deferred value: input response remains immediate
  const list = useMemo(() => {
    return Array.from({ length: 10000 }, (_, i) => `${deferredText} item ${i}`);
  }, [deferredText]);

  return (
    <div>
      <input value={text} onChange={e => setText(e.target.value)} />
      <ExpensiveList list={list} />
    </div>
  );
}

function ExpensiveList({ list }) {
  return (
    <ul>
      {list.slice(0, 50).map((item, index) => <li key={index}>{item}</li>)}
    </ul>
  );
}
```

6. Resource and Form Hooks (React 19)

React 19 introduces hooks specifically designed for actions, resources, and async state mutations.

use

Reads resources (like Promises or Context) inline inside conditional loops and checks.

- **Use Cases:** Resolving async data directly in rendering (suspends the component), or loading conditional context values.
- **How it works:** Unlike other hooks, use can be called inside loops and if statements.

Code Example

```
import { use, Suspense } from 'react';

const fetchMessage = new Promise((resolve) => setTimeout(() => resolve('Hello from React 19!'), 1000));

function MessageComponent() {
  // Resolves the promise. Component suspends until resolved.
  const message = use(fetchMessage);
  return <h2>{message}</h2>;
}

export default function App() {
  return (
    <Suspense fallback={<p>Loading message...</p>}>
      <MessageComponent />
    </Suspense>
  );
}
```

useActionState (React 19 / formerly useFormState)

Hook designed to simplify forms using Actions (asynchronous function submission).

- **Use Cases:** Form submission handling, resolving validation errors, and display of pending/completed states.
- **How it works:** Accepts an action function and initial state. Returns the current state, a wrapped action function (for form actions), and an isPending indicator.

Code Example

```
import { useActionState } from 'react';

async function updateUsername(prevState, formData) {
  const username = formData.get('username');
  try {
    // API mock call
    await new Promise((resolve) => setTimeout(resolve, 1000));
    return { success: true, message: `Username updated to ${username}` };
  } catch (err) {
    return { success: false, message: 'Failed to update username' };
  }
}

function ProfileForm() {
  const [state, formAction, isPending] = useActionState(updateUsername, null);

  return (
    <form action={formAction}>
      <input type="text" name="username" required />
      <button type="submit" disabled={isPending}>
        {isPending ? 'Updating...' : 'Update'}
      </button>
      {state && <p className={state.success ? 'success' :
'error'}>{state.message}</p>}
    </form>
  )
}
```

```
);  
}
```

useFormStatus (React 19)

Retrieves submission status of a parent <form> container.

- **Use Cases:** Creating nested child action components (like specialized Submit buttons) that need to disable themselves during parents' async posts.
- **How it works:** Read state parameters from the nearest parent form. Must be called in a child component inside the <form>.

Code Example

```
import { useFormStatus } from 'react-dom';  
  
function SubmitButton() {  
  const { pending, data, method, action } = useFormStatus();  
  
  return (  
    <button type="submit" disabled={pending}>  
      {pending ? 'Saving...' : 'Save Changes'}  
    </button>  
  );  
}  
  
// Parent component:  
function FormWrapper() {  
  return (  
    <form action={async () => { / async action / }}>  
      <input type="text" name="input" />  
      <SubmitButton /> {/ Hook is called here inside child /}  
    </form>  
  );  
}
```


useOptimistic (React 19)

Displays a temporary state value during async actions, reverting it if the action fails.

- **Use Cases:** Message bubbles, likes/ratings, checklist completions where UI updates instantaneously before API verification completes.
- **How it works:** Takes original state and a reducer-like function. Returns optimistic state value and an updater function.

Code Example

```
import { useOptimistic, useState } from 'react';

function MessageBoard() {
  const [messages, setMessages] = useState([ { text: 'Hello!' } ]);
  const [optimisticMessages, addOptimisticMessage] = useOptimistic(
    messages,
    (state, newMessageText) => [...state, { text: newMessageText, sending: true } ]
  );

  async function sendMessageAction(formData) {
    const messageText = formData.get('message');

    // 1. Immediately apply optimistic UI change
    addOptimisticMessage(messageText);

    // 2. Perform background server action
    await new Promise((resolve) => setTimeout(resolve, 1500)); // Mock API delay

    // 3. Confirm true state update
    setMessages(prev => [...prev, { text: messageText }]);
  }

  return (
```

```

<div>
  {optimisticMessages.map((m, i) => (
    <p key={i} style={{ opacity: m.sending ? 0.5 : 1 }}>
      {m.text} {m.sending && '(sending...)}'
    </p>
  ))}
  <form action={sendMessageAction}>
    <input type="text" name="message" required />
    <button type="submit">Send</button>
  </form>
</div>
);
}

```

7. Miscellaneous Hooks

useId

Generates unique, stable IDs that are compatible with server-side rendering (SSR) and client hydration.

- **Use Cases:** Generating unique HTML element IDs for accessibility bindings (e.g. aria-describedby or labels to inputs).

Code Example

```

import { useId } from 'react';

function FormField() {
  const id = useId();
  return (
    <div>
      <label htmlFor={id}>Username:</label>
      <input id={id} type="text" />
    </div>
  );
}

```

```
);  
}
```

useSyncExternalStore

Subscribes to an external store (like Redux, Zustand, router paths, or browser status API).

- **Use Cases:** Synchronizing non-React libraries or browser APIs (like network status, storage status) to component rendering without tear.
- **How it works:** Takes subscribe function, getSnapshot function, and an optional getServerSnapshot function.

Code Example

```
import { useSyncExternalStore } from 'react';  
  
// Subscribe to browser online/offline status  
function getSnapshot() {  
  return navigator.onLine;  
}  
  
function subscribe(callback) {  
  window.addEventListener('online', callback);  
  window.addEventListener('offline', callback);  
  return () => {  
    window.removeEventListener('online', callback);  
    window.removeEventListener('offline', callback);  
  };  
}  
  
function OnlineIndicator() {  
  const isOnline = useSyncExternalStore(subscribe, getSnapshot);  
  return <p>Status: {isOnline ? '🟢 Online' : '🔴 Offline'}</p>;  
}
```

useDebugValue

Displays custom debug labels in React Developer Tools.

- **Use Cases:** Used within custom Hooks to show developer-facing labels next to Hook states.

Code Example

```
import { useState, useDebugValue } from 'react';

function useFriendStatus(friendID) {
  const [isOnline, setIsOnline] = useState(null);

  // Displays online/offline tag inside React DevTools
  useDebugValue(isOnline ? 'Online' : 'Offline');

  return isOnline;
}
```

Practical Scenarios & Questions for Each Hook

Below are typical development scenarios and interview questions mapped to each hook to help you identify when to use them.

`useState`

- Question: "How do I track form inputs (e.g., name, email, password) dynamically as a user types, and reset them on submit?"
- Solution: Maintain an object state like `const [form, setForm] = useState({ name: "", email: "", password: "" })` and update the respective keys using input `onChange` handlers.

`useReducer`

- Question: "I am building a shopping cart where users can add items, remove items, increase/decrease item quantities, and apply promo codes. My `useState` setters are becoming huge and messy. How can I clean this up?"
- Solution: Implement a central reducer function that processes actions (e.g., `dispatch({ type: 'ADD_ITEM', payload: item })`) and manage it via `useReducer`.

`useContext`

- Question: "I have a global settings configuration (like light/dark mode theme or user authentication status). How can I make it available to deeply nested components without passing props through every intermediate child?"
- Solution: Create a custom context and wrap your tree with a provider, then consume it inside components using `const theme = useContext(ThemeContext)`.

`useRef`

- Question: "How do I focus a text input immediately when a modal opens, or how can I track how many times a component has rendered without triggering a re-render itself?"
- Solution: Store the reference to the DOM element or a counter in a ref: `const countRef = useRef(0)`. Mutate `.current` directly.

`useImperativeHandle`

- Question: "I am writing a reusable custom drawer/modal component. How do I expose imperative methods like `.open()` and `.close()` to the parent component using a ref?"
- Solution: Combine `forwardRef` with `useImperativeHandle` inside the modal component to expose custom callbacks.

`useEffect`

- Question: "How do I load a user profile from an external database API when

the user's ID changes, and ensure old, lagging network calls do not overwrite newer ones?"

- Solution: Trigger the fetch inside `useEffect` with `[userId]` as a dependency, and return a cleanup function to toggle an `active` boolean flag to ignore stale responses.

`useLayoutEffect`

- Question: "I am building a tooltip component. When it opens, it measures the anchor button's dimensions to position itself. In standard `useEffect`, the tooltip appears in the wrong spot for a brief frame and then jumps. How do I prevent this flicker?"
- Solution: Calculate positions synchronously using `useLayoutEffect` before the browser paints the screen.

`useInsertionEffect`

- Question: "I am authoring a custom styling library. How do I dynamically inject `<style>` rules into the DOM before layout calculations occur, minimizing browser style recalculation costs?"
- Solution: Use `useInsertionEffect` to append stylesheet rules onto the document before rendering begins.

`useMemo`

- Question: "My component receives a large list of 5,000 logs and filters them. Typing in a totally unrelated text box causes the list filtering to re-run, causing lag. How do I prevent this?"
- Solution: Wrap the filtering logic in `useMemo` so it recalculates only when either the log list or the filter query changes.

`useCallback`

- Question: "I have a parent component that re-renders. I pass an `onRemove` callback to list items optimized with `React.memo`. Why do the child list items still re-render on every parent render?"
- Solution: Because a new function reference for `onRemove` is created on every parent render. Wrap the function in `useCallback` to maintain reference equality.

`useTransition`

- Question: "I have a live search that filters an extensive database. As I type, the input box is unresponsive and lags behind. How do I keep input typing fast while generating search updates?"

- Solution: Treat the input text state update as high priority, and wrap the database query/list state update in `startTransition` to calculate results in the background.

`useDeferredValue`

- Question: "I am passing a search query down to an expensive third-party graphing component that I can't optimize. How can I defer updating the graph until the user pauses typing?"

- Solution: Create a deferred version of the query: `const deferredQuery = useDeferredValue(query)`, and feed the deferred value into the graph component.

`use`

- Question: "How do I conditionally consume context values or fetch data directly inside nested loops or `if` statements in React 19?"

- Solution: Use the `use(Resource)` hook. It resolves promises or context values and can be called conditionally, unlike traditional React hooks.

`useActionState`

- Question: "How do I handle a form submission that calls an asynchronous API, automatically tracking when it's pending (loading state) and storing any validation errors returned by the server?"

- Solution: Bind the form to an Action using `const [state, formAction, isPending] = useActionState(mySubmitAction)`.

`useFormStatus`

- Question: "I have a reusable submit button component nested inside a complex form. How can this button know if the parent form is currently submitting, without passing loading props down?"

- Solution: Call `const { pending } = useFormStatus()` inside the button component. It will auto-detect the status of the closest parent form.

`useOptimistic`

- Question: "How do I implement an instant 'Like' button or messaging bubble

that updates immediately in the UI while the background server request is still running?"

- Solution: Use `useOptimistic` to show the expected successful state instantly. If the action succeeds, state updates permanently; if it fails, it rolls back.

`useId`

- Question: "I am building accessible form components. How can I match `<label htmlFor={...}>` with an input's `<id>` ensuring that the generated ID matches between the server and the client during hydration?"
- Solution: Call `const id = useId()` to generate a stable, collision-free identifier.

`useSyncExternalStore`

- Question: "I am subscribing to a custom event bus, window width, or Redux store. How do I read values from it dynamically in React without risking 'tearing' (inconsistent UI) during concurrent rendering?"
- Solution: Register the store and retrieve snapshots via `useSyncExternalStore(subscribe, getSnapshot)`.

`useDebugValue`

- Question: "I am creating a custom hook named `useGeolocation`. How do I format and display its internal state values (like 'Lat: 45.2, Lon: 12.3') inside the React Developer Tools extension?"
- Solution: Add `useDebugValue(location ? `Lat: ${location.lat}` : 'Loading')` inside your custom hook.

CustomHooks

Spread and Rest

The Spread and Rest Operators in JavaScript (...)

In JavaScript (and React), the three dots syntax ... performs two opposite operations depending on **where** and **how** it is used: **Spread** and **Rest**.

1. Quick Summary

- **Spread Operator:** Unpacks elements (spreads them out). It takes an array or object and breaks it down into individual components.
- **Rest Parameter:** Packs elements (gathers them up). It collects multiple individual elements and condenses them into a single array or object.

2. The Spread Operator (Unpacking)

Spread is used to duplicate, merge, or convert collections into separate items.

A. Spread with Arrays

```
const fruits = ['apple', 'banana'];  
const veggies = ['brocoli', 'carrot'];  
  
// 1. Copying an array  
const fruitsCopy = [...fruits]; // ['apple', 'banana']  
  
// 2. Concatenating / Merging arrays  
const food = [...fruits, ...veggies, 'pizza'];  
// Result: ['apple', 'banana', 'brocoli', 'carrot', 'pizza']  
  
// 3. Passing array items as separate arguments to a function  
const numbers = [5, 12, 8];  
const max = Math.max(...numbers); // Equivalent to Math.max(5, 12, 8) -> returns 12
```

B. Spread with Objects (Crucial for React State)

```
const user = { name: 'Sarah', role: 'Admin' };  
  
// 1. Copying and adding/modifying properties  
const updatedUser = { ...user, age: 25, role: 'Superadmin' };  
// Result: { name: 'Sarah', role: 'Superadmin', age: 25 }
```

```
// 2. Merging two objects
const details = { city: 'Boston' };
const fullProfile = { ...user, ...details };
// Result: { name: 'Sarah', role: 'Admin', city: 'Boston' }
```

[!WARNING] Both Array and Object Spread perform a **shallow copy**. If the original object contains nested objects/arrays, they are copied by **reference** (not deep value). Changing nested objects in the copy will affect the original.

```
const user = { name: 'Alex', info: { age: 20 } };
const copy = { ...user };
copy.info.age = 21; // Modifies BOTH user and copy because copy.info is still pointing to the same nested address!
```

3. The Rest Operator (Packing / Gathering)

Rest gathers multiple separate values into a single compound structure (an array or object).

A. Rest as Function Parameters

When defined in a function's argument list, it collects all remaining arguments into a clean array.

```
// Gathers any number of numbers into a 'nums' array
function sumEverything(multiplier, ...nums) {
  // nums = [1, 2, 3]
  return nums.reduce((total, n) => total + n, 0) * multiplier;
}

console.log(sumEverything(10, 1, 2, 3));
// Output: (1 + 2 + 3) * 10 = 60
```

*Note: The rest parameter must always be the **last** parameter in the function signature.*

B. Rest in Array Destructuring

```
const colors = ['red', 'green', 'blue', 'yellow'];

// Extract the first color, and gather the rest
const [primary, ...remainingColors] = colors;

console.log(primary);      // 'red'
console.log(remainingColors); // ['green', 'blue', 'yellow']
```

C. Rest in Object Destructuring (Often used in React props)

```
const buttonConfig = { label: 'Submit', color: 'blue', size: 'large', disabled: false };

// Destructure the label, and collect all other style/settings configs
const { label, ...styleProps } = buttonConfig;

console.log(label);    // 'Submit'
console.log(styleProps); // { color: 'blue', size: 'large', disabled: false }
```

4. Visual Comparison for Students

You can visualize the operations like this:

Spread (Unpacking Box)

```
Array: [ 'A', 'B' ]
      \ /
Spread:  A , B    (Individual elements released)
```

Rest (Packing Items into Box)

```
Elements:  A , B
           \ /
Rest:      [ 'A', 'B' ] (Collected into a container)
```

React Props

React Props

1. What Are Props?

- **Props** (short for *properties*) are readonly values passed from a parent component to a child component.
- They are the **only** way to make a component dynamic and reusable.
- In JSX a prop looks like an HTML attribute:

```
<Child name="Alex" age={30} />
```

- Inside Child the values are received via the function argument:

```
function Child({ name, age }) {  
  return <p>{name} is {age} years old.</p>;  
}
```

2. Basic Syntax & Destructuring

```
// Parent.jsx  
import Child from './Child';  
  
export default function Parent() {  
  const user = { name: 'ayush', age: 21, section: 'A' };  
  return <Child {...user} />;  
}  
  
// Child.jsx  
export default function Child({ name, age }) {  
  return (  
    <div>  
      <h2>{name}</h2>  
      <p>Age: {age}</p>  
    </div>  
  );  
}
```

- Using **object spreading** (`{...user}`) is a concise way to pass many props at once.
- Destructuring in the function signature keeps the component tidy and avoids props.name repetition.

3. Default Props

When a parent does not provide a prop, you can give it a fallback value.

3.1 With Function Default Parameters (recommended in modern React)

```
function Greeting({ message = 'Hello, world!', name = 'Guest' }) {  
  return <p>{message} – {name}</p>;  
}
```

3.2 Using defaultProps (legacy, works with class components)

```
function Button({ label }) {  
  return <button>{label}</button>;  
}
```

```
Button.defaultProps = {  
  label: 'Click me',  
};
```

Note: For functional components, default parameters are clearer and typefriendly.

4. Children – The Special props.children

children is a builtin prop that contains whatever you nest between a component's opening and closing tags.

```
function Card({ title, children }) {  
  return (  
    <div className="card">  
      <h3>{title}</h3>  
      <div className="content">{children}</div>  
    </div>  
  );  
}
```

```
// Usage  
<Card title="Profile">  
  <p>Some descriptive text.</p>  
    
</Card>
```

- children can be a single node, an array of nodes, or even a render function.

5. Prop Types – Runtime Validation (JavaScript)

React ships with the prop-types package for declarative type checking.

```
npm install prop-types

import PropTypes from 'prop-types';

function Profile({ name, age, isAdmin, onLogout }) {
  return (
    <div>
      <h2>{name}</h2>
      <p>{age} years old</p>
      {isAdmin && <button onClick={onLogout}>Logout</button>}
    </div>
  );
}

Profile.propTypes = {
  name: PropTypes.string.isRequired,
  age: PropTypes.number,
  isAdmin: PropTypes.bool,
  onLogout: PropTypes.func,
};

Profile.defaultProps = {
  age: 0,
  isAdmin: false,
  onLogout: () => {},
};
```

- `isRequired` raises a console warning if the prop is missing.
- Helpful during development, but strips out in production builds.

6. Props with TypeScript – Compiletime Safety

If your project uses TypeScript, define a Props interface or type.

```
import React from 'react';

type ButtonProps = {
  label: string;
  disabled?: boolean;
  onClick: (event: React.MouseEvent<HTMLButtonElement>) => void;
```

```

};

const Button: React.FC<ButtonProps> = ({ label, disabled = false, onClick }) => (
  <button disabled={disabled} onClick={onClick}>
    {label}
  </button>
);

export default Button;

```

- Optional props are marked with ?.
- Default values can still be set via function defaults (disabled = false).
- TypeScript will catch mismatched prop names **before** the code even runs.

7. Passing Callback Functions (Event Handlers)

A common pattern is to give a child a function it can call back to the parent.

```

// Parent.jsx
function Parent() {
  const [count, setCount] = useState(0);

  const increment = () => setCount(c => c + 1);

  return (
    <div>
      <h1>Count: {count}</h1>
      <CounterButton onClick={increment} />
    </div>
  );
}

// CounterButton.jsx
function CounterButton({ onClick }) {
  return <button onClick={onClick}>+1</button>;
}

```

- The parent owns the state, the child only triggers an action.
- **Never modify a parent's state directly inside a child** – always go through a callback.

8. Rest Props (...rest) – Forwarding Unknown Props

When you want a component to accept *any* extra attributes (e.g., `className`, `data-*`, `aria-*`), collect them with rest syntax and spread them onto a DOM element.

```
function Input({ label, ...rest }) {  
  return (  
    <label>  
      {label}  
    <input {...rest} />  
    </label>  
  );  
}
```

```
// Usage – all extra props are passed to the <input>  
<Input label="Email" type="email" placeholder="you@example.com" required />
```

- This pattern makes components flexible and integrates nicely with libraries like **styledcomponents** or **MUI**.

9. Advanced Pattern – Render Props

A *render prop* is a function prop that returns JSX. It lets a component decide **what** to render while the parent supplies **how**.

```
function DataFetcher({ url, children }) {  
  const [data, setData] = useState(null);  
  const [loading, setLoading] = useState(true);  
  
  useEffect(() => {  
    fetch(url)  
      .then(r => r.json())  
      .then(json => {  
        setData(json);  
        setLoading(false);  
      });  
  }, [url]);  
  
  return children({ data, loading }); // <-- children is a function  
}  
  
// Usage  
<DataFetcher url="/api/user">  
  {({ data, loading }) =>  
    loading ? <p>Loading...</p> : <pre>{JSON.stringify(data, null, 2)}</pre>  
  }  
</DataFetcher>
```

- This is a flexible alternative to higherorder components (HOC).

10. Common Pitfalls & Best Practices

Pitfall	Why It's Bad	Fix
Mutating prop objects (props.user.age = 30)	Props are readonly; mutating breaks the contract and can cause stale UI.	Treat props as immutable; derive new objects with spread ({...props.user, age: 30}).
Passing new function objects on every render (onClick={() => doSomething()}))	Triggers unnecessary rerenders of child components using React.memo.	Define callbacks with useCallback or move them outside the render.
Forgetting key when rendering list of children	React cannot reconcile items correctly, leading to bugs.	Provide a stable key (key={item.id}).
Using defaultProps with functional components	Default parameters are clearer; defaultProps may be stripped in future TSX.	
Overusing ...rest on a component that also defines explicit props	Can unintentionally forward unwanted attributes to the DOM, causing warnings.	
Missing PropTypes/TS definitions	Harder to debug and no autocompletion.	

Passing primitive values that change often (e.g., objects created inline)	Causes rerenders because the reference changes each render.	Memoise objects with <code>useMemo</code> .
--	---	---

Recap

- Props are immutable inputs from parent to child.
- Use destructuring, default values, and **spread** (...) for clean APIs.
- Validate with **PropTypes** or **TypeScript**.
- Pass callbacks for child to parent communication.
- Leverage children, render props, and rest props for advanced composition.
- Follow best practice guidelines to avoid common bugs and keep your UI performant.

Prop Drilling

Prop Drilling

What is Prop Drilling?

Prop drilling is a pattern in React where **data (props) is passed down through multiple levels of components** that do not need the data themselves, solely to reach a deeply nested child that does.

When a parent component holds a piece of state that a grandchild needs, the typical React approach is to pass that state (or a handler) as a prop to the immediate child, which in turn forwards it to its child, and so on. This creates a *chain of props* that can become hard to maintain as the component tree grows.

Simple Code Example (Prop Drilling)

```
// App.jsx (Parent)
import React, { useState } from "react";
import { ChildA } from "../ChildA";

export const App = () => {
  const [theme, setTheme] = useState("light");

  return (
    <div>
      <h1>Current theme: {theme}</h1>
      { /* Pass theme and setter down the tree */ }
      <ChildA theme={theme} setTheme={setTheme} />
    </div>
  );
};

// ChildA.jsx (intermediate component)
import React from "react";
import { ChildB } from "../ChildB";

export const ChildA = ({ theme, setTheme }) => (
  <section>
    { /* This component does not use the props, it merely forwards them */ }
    <ChildB theme={theme} setTheme={setTheme} />
  </section>
);
```



```
// ChildB.tsx (intermediate component)
import React from "react";
import { GrandChild } from "../GrandChild";

export const ChildB = ({ theme, setTheme }) => (
  <div>
    {/* Still no direct usage – forwarding continues */}
    <GrandChild theme={theme} setTheme={setTheme} />
  </div>
);

// GrandChild.jsx (the real consumer)
import React from "react";

export const GrandChild = ({ theme, setTheme }) => (
  <button
    onClick={() => setTheme(theme === "light" ? "dark" : "light")}
    style={{
      background: theme === "light" ? "#fff" : "#333",
      color: theme === "light" ? "#000" : "#fff",
    }}
  >
    Toggle Theme (currently {theme})
  </button>
);
```

In the example above, **ChildA** and **ChildB** never read `theme` or `setTheme`; they are merely conduits. If the component hierarchy changes, you have to update every intermediate component's signature, which quickly becomes noisy.

Why Prop Drilling Can Be Problematic

Issue	Description
Verbosity	Every intermediate component's prop list grows, even if it doesn't need the data.

Tight Coupling	Child components become dependent on the exact shape of the parent's props, making refactoring risky.
Scalability	Deep trees with many shared pieces of state lead to long chains of props, hurting readability.
Maintenance Overhead	Adding a new consumer often requires editing many unrelated components.

Strategies to Mitigate Prop Drilling

1. Lift State Up to a Common Ancestor

If several sibling branches need the same data, place the state in their nearest common ancestor and pass it only to the branches that truly need it. This reduces the depth of the prop chain but does **not** eliminate it when the consumer is deeply nested.

2. React Context API

The Context API creates a *globalish* store that any descendant can subscribe to **without** passing props through every level.

Context Example

```
// ThemeContext.tsx
import React, { createContext, useState, useContext } from "react";

const ThemeContext = createContext({
  theme: "light" as "light" | "dark",
  toggle: () => {}
});

export const ThemeProvider: React.FC<{ children: React.ReactNode }> = ({ children }) => {
  const [theme, setTheme] = useState("light");
  const toggle = () => setTheme(t => (t === "light" ? "dark" : "light"));

  return (
```

```

    <ThemeContext.Provider value={{ theme, toggle }}>
      {children}
    </ThemeContext.Provider>
  );
};

export const useTheme = () => useContext(ThemeContext);

// App.tsx (root)
import React from "react";
import { ThemeProvider } from "../ThemeContext";
import { ChildA } from "../ChildA";

export const App = () => (
  <ThemeProvider>
    <ChildA />
  </ThemeProvider>
);

// GrandChild.tsx (consumer, no prop drilling!)
import React from "react";
import { useTheme } from "../ThemeContext";

export const GrandChild = () => {
  const { theme, toggle } = useTheme();
  return (
    <button
      onClick={toggle}
      style={{
        background: theme === "light" ? "#fff" : "#333",
        color: theme === "light" ? "#000" : "#fff",
      }}
    >
      Toggle Theme (currently {theme})
    </button>
  );
};

```

Notice that **ChildA** and **ChildB** no longer receive any props related to theme; the grandchild pulls the data directly from the context.

3. StateManagement Libraries (Redux, Zustand, Jotai, Recoil...)

When an application grows beyond a few pieces of state, a dedicated store can provide a *single source of truth* and selectors that any component can subscribe to. These libraries also give you powerful devtools, timetravel debugging, and middleware.

Minimal Zustand Example

```
// store.ts
import create from "zustand";

export const useStore = create(set => ({
  theme: "light" as "light" | "dark",
  toggleTheme: () => set(state => ({ theme: state.theme === "light" ? "dark" : "light" })),
}));

// GrandChild.tsx
import React from "react";
import { useStore } from "../store";

export const GrandChild = () => {
  const theme = useStore(state => state.theme);
  const toggleTheme = useStore(state => state.toggleTheme);

  return (
    <button
      onClick={toggleTheme}
      style={{
        background: theme === "light" ? "#fff" : "#333",
        color: theme === "light" ? "#000" : "#fff",
      }}
    >
      Toggle Theme (currently {theme})
    </button>
  );
};
```

The store lives outside the component tree, eliminating any need for prop propagation.

4. Custom Hooks for Shared Logic

If the only reason you are passing a function down is to *trigger* an effect, you can encapsulate that logic in a custom hook that each component can call directly.

```
// useThemeToggle.ts
import { useStore } from "../store"; // could also use Context
export const useThemeToggle = () => {
  const toggle = useStore(state => state.toggleTheme);
  return toggle;
};
```

Then GrandChild simply calls `const toggle = useThemeToggle();` – no props involved.

When Is Prop Drilling Acceptable?

- **Very shallow component trees** (12 levels) where the intent is explicit.
- **Oneoff data** needed by a single child; introducing a context may feel heavyweight.
- **Performancecritical paths** where you want to avoid the extra render cycles caused by context subscription (though React's context is highly optimized).

If the hierarchy is deeper or the same data is required in multiple distant branches, moving to Context or a store is almost always the cleaner approach.

Quick Checklist to Detect Prop Drilling

1. Do any intermediate components receive props they never use?
2. Does a prop need to travel more than **two** levels?
3. Are you frequently updating the prop list on many components when the shape of the data changes?
4. Do multiple unrelated branches need the same piece of state?

If you answered yes to any of the above, consider refactoring with Context or a statemanagement library.

Summary

- **Prop drilling**: passing props through components that don't need them.
- **Problems**: verbosity, tight coupling, scalability issues.
- **Solutions**:
 - Lift state up to the nearest common ancestor.
 - Use **React Context** for globalish data.
 - Adopt a **statemanagement library** (Redux, Zustand, etc.) for larger apps.
 - Extract shared logic into **custom hooks**.

SPA Routing

ContextAPI

